Robust Regression Engine

Theory and Project Explanation – House Price Prediction

1. introduction

In this project i built a house price prediction system for a real estate company. The company already had a simple regression model, but it overfitted and gave unstable predictions on new data. So my goal was to create a more robust pipeline that works well on data it has never seen before.

To do this i used regularization, proper cross-validation, tree-based models and Support Vector Regression. i compared all of them on the same train–test split and the same metrics, and then picked the best one using validation performance. At the end, i saved the best model and made a Streamlit app so anyone can enter property details and get a predicted price.

2. Part A: Theory Concepts

2.1 What is regularization and why is it needed?

Regularization means adding a penalty to the loss function so that the model cannot make its coefficients too big. Without it, a model can learn the noise in the training data and then perform badly on new data, which is called overfitting. i needed regularization because plain linear regression can become unstable when features are correlated, for example area, bedrooms and bathrooms in my dataset.

2.2 Ridge (L2) vs Lasso (L1)

Both add a penalty, but in different ways. Ridge adds the sum of squared coefficients. it makes all coefficients smaller but never exactly zero, and it works well when features are correlated. Lasso adds the sum of absolute coefficients. it can push weak coefficients to exactly zero, so it also works like automatic feature selection. in both, the strength of the penalty is controlled by alpha. A bigger alpha means a simpler model.

2.3 What is cross-validation and why is it important?

in cross-validation i split the training data into parts (folds). The model trains on some folds and is checked on the remaining fold, and this repeats until every fold has been used for checking once. The final score is the average. This is better than one single split because one split can be lucky or unlucky. i used it for choosing hyperparameters and for checking overfitting without touching the test set.

2.4 Cross-validation techniques

- K-Fold: the data is divided into K equal folds (i used K = 5). Each fold is used for validation once.
- Stratified K-Fold: each fold keeps the same distribution of the target. it is normally for classification, so for regression i divided the price into 5 quantile bins and stratified on those bins.
- Leave-One-Out (LOOCV): each single row is the validation set once and the model trains on all the other rows. it is almost unbiased but very slow for big models.
- Time Series Split: the data is sorted by time and the model only trains on the past and tests on the future. it avoids data leakage when the data has a time order. i used sale_year for this.

2.5 Why are tree-based models less sensitive to feature scaling?

A decision tree splits on one feature at a time using a threshold, like area_sqft ≤ 1500. Scaling only changes the numbers in a monotonic way, so the order of values and the best split stay the same. Trees do not use

distances or gradients, so they do not need scaling. Ridge, Lasso and SVR do need it, so i scaled the data for those models only.

3. Part B: Dataset and Preparation

The dataset has 3,800 properties sold between 2010 and 2023, with 12 columns and no missing values. it contains property size and structure (area, bedrooms, bathrooms), location indicators (location score, distance from city, near school, near metro, crime rate index), property age and the sale date. The target variable is house_price_inr.

I used 10 features: area_sqft, bedrooms, bathrooms, location_score, property_age, distance_city_km, near_school, near_metro, crime_rate_index and sale_year. i created sale_year from sale_date and dropped property_id because it is only an identifier. For easier reading i converted the price to Lakh iNR. i split the data 80/20, giving 3,040 rows for training and 760 rows for testing.

For preprocessing, i put StandardScaler inside a Pipeline. This way the scaler is fitted only on the training folds during cross-validation, so no information leaks from the validation data. From the correlation heatmap i saw that price is right-skewed and that area and location score have the strongest positive effect, while property age and crime rate pull the price down.

4. Part C: Regularized Linear Models

I built Ridge and Lasso and tuned alpha using 5-fold cross-validation over 30 values from 0.001 to 1000. The best alpha was about 2.04 for Ridge and 0.117 for Lasso. The alpha vs error plot shows that a very high alpha over-shrinks the coefficients and the error goes up (underfitting), while a very low alpha behaves like normal linear regression.

Model	Best alpha	Training RMSE	Validation RMSE	Test RMSE
Ridge (L2)	2.0434	24.8247	24.9512	25.3965
Lasso (L1)	0.1172	24.8266	24.9493	25.4201

The two models gave almost the same error. Looking at the coefficients, area_sqft (about 69.5) and location_score (about 36.8) are by far the biggest drivers, and property_age is negative (about −6.5). Lasso did not make any feature exactly zero here, because all features carry some information and the dataset is large. it did shrink the weak ones like near_school from 0.154 to 0.034. My honest conclusion is that regularization did not improve accuracy much on this dataset, but it keeps the coefficients stable.

5. Part D: Cross-Validation Strategies

i applied all four techniques to the tuned Ridge model and compared them.

Strategy	Folds	RMSE	RMSE std	MAE	R^2
K-Fold (k=5)	5	24.951	0.890	19.066	0.9163
Stratified K-Fold	5	24.899	0.788	19.032	0.9168
Time Series Split	5	24.850	0.919	18.948	0.9167
LOOCV	3040	24.923	–	19.039	0.9167

All four strategies gave very similar scores (RMSE between 24.85 and 24.95), which tells me the estimate is stable and does not depend on one lucky split. Stratified K-Fold had a slightly smaller spread between folds because every fold covers all price ranges. LOOCV is accurate but needs one model fit per row, so it would be slow for bigger models. The Time Series Split is the most realistic if the model will predict future sales, because it only learns from the past. i used 5-fold CV for tuning because it is fast and stable, and the Time Series Split as an extra check.

6. Part E: Tree-Based Models

First i trained an unpruned decision tree. it reached a train R^2 of 1.0000 (depth 21, 3,040 leaves, one leaf per training row), but the test R^2 was only 0.8692. This is a clear example of overfitting. Then i controlled the complexity with max_depth, min_samples_split and min_samples_leaf using grid search. The best tree had max_depth = 8 and min_samples_leaf = 20, and the test R^2 improved to 0.9071.

Next i trained a Random Forest with 150 trees (max_features = 0.6, min_samples_leaf = 3). Because it averages many different trees, the variance goes down. its test RMSE was 23.87 compared with 27.35 for the single tree. The forest is more accurate, but the single tree is easier to explain. Random Forest feature importance also showed area_sqft and location_score as the top features, which matches what Ridge and Lasso told me.

Model	Train RMSE	Test RMSE	Train R^2	Test R^2
Decision Tree (unpruned)	0.0000	32.4529	1.0000	0.8692
Decision Tree (tuned)	22.9116	27.3488	0.9296	0.9071
Random Forest (tuned)	13.6275	23.8732	0.9751	0.9292

7. Part F: Support Vector Regression

SVR tries to fit a function inside an epsilon tube and only penalizes points outside of it. C controls how strongly errors are penalized, gamma controls how far the influence of one sample reaches in the RBF kernel, and epsilon is the width of the tube. Because SVR depends on distances, i scaled the features, and i also standardized the target using TransformedTargetRegressor so that epsilon and C make sense.

I tried a linear kernel and an RBF kernel and tuned them with grid search. The best linear SVR had C = 1 and epsilon = 0.2 (CV RMSE 24.96). The best RBF SVR had C = 1, gamma = 0.03 and epsilon = 0.05 (CV RMSE 22.68). The linear SVR behaved like the Ridge model, while the RBF kernel could capture non-linear patterns in price and beat every other model.

8. Part G: Model Comparison and Evaluation

i evaluated every model with MSE, MAE, RMSE and R^2. Lower error is better and higher R^2 is better.

Model	Test RMSE	Test MAE	Test R^2	CV RMSE
Linear Regression (baseline)	25.3915	19.4488	0.9199	24.9514
Ridge (L2)	25.3965	19.4463	0.9199	24.9512
Lasso (L1)	25.4201	19.4611	0.9198	24.9493
Decision Tree (tuned)	27.3488	20.2227	0.9071	27.3312
Random Forest (tuned)	23.8732	17.5312	0.9292	23.5698

Model	Test RMSE	Test MAE	Test R^2	CV RMSE
SVR Linear (tuned)	25.5235	19.4472	0.9191	24.9585
SVR RBF (tuned)	22.7372	16.6468	0.9358	22.6845

Comparing the groups, SVR had the best average (RMSE 24.13, R^2 0.9275), then the regularized linear models (RMSE 25.41) and then the tree-based models (RMSE 25.61). The average of trees is pulled down by the single decision tree, but the Random Forest alone is the second best model.

Overfitting and underfitting

- Ridge, Lasso, Linear Regression and SVR Linear: train and test R^2 are almost equal (about 0.917 and 0.919), so there is no overfitting. They are a bit simple and cannot capture non-linear patterns.
- Decision Tree (unpruned): train R^2 1.0 vs test 0.869, a clear case of overfitting. After tuning the gap came down to about 0.02.
- Random Forest: the biggest gap among the tuned models (train 0.975 vs test 0.929, gap 0.046). it is a mild overfit, but it still generalizes well.
- SVR RBF: train R^2 0.938 and test R^2 0.936, so the gap is only 0.003. This is the best balance between bias and variance.

9. Part H: Final Analysis

Best model and justification

The best model is SVR with the RBF kernel. it has the lowest cross-validated RMSE (22.68 Lakh) and the lowest test error (RMSE 22.74 Lakh, MAE 16.65 Lakh, R^2 0.9358). its train and test scores are almost the same, so it generalizes well to unseen data.

impact of regularization

For this dataset, regularization gave stable but not much better results than the baseline (CV RMSE 24.951 for Linear, Ridge and Lasso). This is because the data is large and there are only 10 features. The bigger effect of the same idea was seen in trees, where limiting the depth improved the test R^2 from 0.869 to 0.907.

Role of cross-validation

Cross-validation helped me choose alpha, tree depth, C, gamma and epsilon without using the test set. The four CV strategies gave almost the same RMSE (24.85 to 24.95), so the results are stable.

Linear vs non-linear regressors

The best linear model (Ridge, test RMSE 25.40) was beaten by the best non-linear model (SVR RBF, test RMSE 22.74). This tells me that house prices have some non-linear behavior, such as interaction between area and location, which linear models cannot capture.

Business interpretation

The model has a typical error (MAE) of about ₹16.6 lakh per house, which is about 8% of the average price of around ₹2.05 crore. it explains about 93.6% of the price variation. This is good enough to support valuation, suggest listing prices and find houses that look over-priced or under-priced. The biggest price drivers are area and location score, so the company should focus on those. The model should be retrained regularly because the market changes with time.

10. Streamlit App and Conclusion

I saved the best model as final_model.joblib and built a Streamlit app (app.py). The user enters area, bedrooms, bathrooms, location score, age, distance, crime rate, school, metro and sale year, and the app shows the predicted price along with a likely range based on the model error. it also has a tab that shows the model comparison tables.

To conclude, this project showed me that a good model is not the one that has the lowest training error, but the one that performs well on unseen data. Using a pipeline, cross-validation and hyperparameter tuning gave a fair comparison of all models, and the SVR RBF model was the most reliable one. in the future i can add more features like locality and amenities, try Gradient Boosting, and monitor price drift over time.